Database Management Systems

Full Text Search in MySQL

Prof. Mahitha G
Department of Computer Science and Engineering

Acknowledgment: Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering

Full Text Search in MySQL

Full-Text Search is a feature in MySQL where you can search for words or phrases in large text-based columns (CHAR, VARCHAR, TEXT) using natural language processing (NLP). Instead of just checking if a string is present, full-text search **looks for relevant matches** and **ranks results based on how well they match the search terms**.

Full-Text Search in MySQL allows you to perform advanced text-based searches on columns with textual data. It is particularly useful for searching large text fields like articles, descriptions, or comments. It is more powerful than **LIKE/REGEXP**, since it ranks results by relevance.

Features -

- **Full-Text Indexes** → much faster than **LIKE**
- **Search Modes** → Natural Language, Boolean, Query Expansion
- **Relevance Ranking** → Scores based on closeness of match
- Handles stopwords, synonyms, and word proximity.

Full Text Search in MySQL

The key feature of Full Text Search is that it uses the **MATCH(column) AGAINST('search terms')** syntax to search.

Suppose you have a table called ‘Product’ with different columns such as product_id, product_name, description and price. Suppose you want to search for the products that have "smartphone" in their name or description. The SQL query for that would be :

```
SELECT * FROM products
WHERE MATCH(product_name, description)
AGAINST('smartphone' IN BOOLEAN MODE);
```

Here,
MATCH function is used to perform full text search on product_name and description column.

AGAINST specifies the search term "smartphone" and in boolean mode.

Full Text Search vs Normal String Search

Feature	LIKE/REGEXP	FULL TEXT SEARCH
Index Usage	Does not use indexes effectively. Slow for large data	Uses special indexes Faster searches
Relevance Ranking	No relevance ranking, just returns matches	Results ranked by relevance
Natural Language Support	Not supported	Supported (Natural Language Mode)
Word Boundary Detection	Not accurate (matches substrings like “cat” in “concatenate”)	Accurate word boundary detection
Boolean Logic	Not supported	Supported (AND, OR, NOT operators)
Phrase Searching	Possible but imprecise	Supported and more accurate

Working of Full Text Search

By default or with the **IN NATURAL LANGUAGE MODE** modifier, the **MATCH()** function performs a natural language search for a string against a text collection.

A collection is a set of one or more columns included in a FULLTEXT index. The search string is given as the argument to **AGAINST()**.

For each row in the table, MATCH() returns a relevance value; that is, a similarity measure between the search string and the text in that row in the columns named in the MATCH() list.

Working :

1. Use columns of type VARCHAR or TEXT
2. Create a **FULLTEXT INDEX** on the column(s)
3. Use MATCH(column1, column2) with AGAINST('word')

Database Management Systems

Working of Full Text Search

Example :

```
CREATE TABLE posts (
   id INT UNSIGNED AUTO_INCREMENT NOT NULL PRIMARY KEY,
   title VARCHAR(200),
   body TEXT
) ENGINE=InnoDB;

ALTER TABLE posts
ADD FULLTEXT(title, body); -- To enable FTS

INSERT INTO posts (title, body) VALUES
('MySQL Tutorial', 'Learn the basics of database systems'),
('Advanced Database', 'We explore indexing and performance'),
('Cooking Tips', 'Always use fresh ingredients'),
('MySQL vs PostgreSQL', 'Comparison of two database systems'),
('Oracle Tips', 'Learn how to use Oracle effectively');
```

Database Management Systems

Working of Full Text Search

PES UNIVERSITY

Query : Select those rows where either the body or the title have the word ‘database’

SELECT * FROM posts
WHERE MATCH (title, body)
AGAINST ('database' IN NATURAL LANGUAGE MODE);

```
mysql> SELECT * from posts
    -> WHERE MATCH (title, body)
    -> AGAINST ('database' IN NATURAL LANGUAGE MODE);
+----+--------------------+-----------------------------------+
| id | title              | body                              |
+----+--------------------+-----------------------------------+
|  1 | MySQL Tutorial     | Learn the basics of database systems |
|  2 | Advanced Database  | We explore indexing and performance  |
|  4 | MySQL vs PostgreSQL | Comparison of two database systems   |
+----+--------------------+-----------------------------------+
3 rows in set (0.00 sec)
```

Working of Full Text Search

Explanation :

- We perform a **full-text search** in **natural language mode** for the word "database".
- It searches the title and body fields for relevance to the term "database".
- Returns rows where the relevance score is **non-zero** (i.e., the word "database" appears and is significant).

Natural Language Mode :

1. Uses natural language processing to rank results based on relevance.
2. Ignores **stopwords** (like "a", "the", etc.) and very short words (less than 4 characters, by default).
3. **Does not require special syntax** (unlike BOOLEAN mode).

Modes of Full Text Search

Mode	Description
Natural Language Mode	Default mode; ranks relevance
Boolean Mode	Allows operators (+, -, *) for complex queries

Relevance

In MySQL Full-Text Search, **Relevance** is a numeric score that tells **how closely a row matches the search terms**.

It's a **floating-point number** (0.0, 0.25, 1.2, etc.)

Higher Score = Better Match

Relevance is calculated using:

1. How often the search term appears in the column
2. How rare the term is across all rows.
3. Length of the document (shorter docs = higher relevance if term is present)

Relevance

Example :

```
SELECT id, title,
      MATCH(title, body) AGAINST('database') AS relevance
FROM posts
WHERE MATCH(title, body) AGAINST('database')
ORDER BY relevance DESC;
```

What it does:

1. Selects matching rows which match against the word ‘database’
2. Shows the relevance score
3. Orders results by relevance descending (highest match first)

Now: What If You Don't Mention Relevance?

SELECT id, title FROM posts WHERE MATCH(title, body) AGAINST('database');

What happens:
Results are still matched using full-text search.
MySQL will still use the **FULLTEXT index**.

By default, **MySQL will sort the results by relevance**, **if and only if**:

1. You **don't use any ORDER BY**.
2. You're **using MATCH(...) AGAINST(...) in the WHERE clause**.
3. The **FULLTEXT index** is being used (not bypassed).
4. It's a **simple query (no joins)**.

So yes — even **without ORDER BY or showing relevance**, the **rows will still be sorted by highest relevance**, **automatically**, under those conditions.

Relevance

Important Note: If you **add any other ORDER BY clause**, MySQL will stop auto-sorting by relevance.

```
SELECT id, title FROM posts WHERE MATCH(title, body) AGAINST('database')
                    ORDER BY id; --  Will sort by id, not relevance
```

Boolean Mode

What is BOOLEAN MODE?

BOOLEAN MODE allows you to:

1. Use **operators** (+, -, ", *, etc.) to include, exclude, or group terms.
2. Search for **partial words** using wildcards (*).
3. Write **complex search logic** (AND, OR, NOT-like behavior).
4. Search even if the word is a **stopword** or appears in **every row**.

You must **explicitly write**:
AGAINST ('your_query' IN BOOLEAN MODE)

Syntax Example

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+mysql -database' IN BOOLEAN MODE);
-- Here, we are including ‘mysql’ and excluding ‘database;

Boolean Mode

Some Common Boolean Operators

Operator	Meaning	Example
+	Word must be present	+mysql (must contain "mysql")
-	Word must not be present	-database (must not contain "database")
(none)	Word is optional, but affects relevance	database-row with the given word are ranked higher
""	Search exact phrase	"mysql tutorial"
*	Wildcard (suffix only)	data* (matches "data", "database")
( )	Group terms	+(mysql database) –must contain mysql or database

PES UNIVERSITY

Example Queries in Boolean Mode

1. Include one word, exclude another

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+mysql -oracle' IN BOOLEAN MODE);

Must contain "mysql", must **not** contain "oracle"

2. Exact Phrase Match

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('"database systems"' IN BOOLEAN MODE);

Must contain the **exact phrase** ‘database systems’

PES UNIVERSITY

3. Prefix Search with Wildcard

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('data*' IN BOOLEAN MODE);

Matches: "data", "database", "databases", etc.

Only suffix wildcards are supported (e.g., data*), not *data.

4. Grouping Words

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+(mysql database) -oracle' IN BOOLEAN MODE);

Database Management Systems

Boolean Mode - Practical Exercise

Here's a practical exercise on **Full-Text Search in MySQL** using Boolean and Natural Language Modes. It has :

- A sample table 'articles'
- Inserted data
- Practice questions ranging from basic to advanced

CREATE TABLE articles (article_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255), content TEXT, FULLTEXT (title, content));

INSERT INTO articles (title, content) VALUES
('Introduction to Databases', 'Databases are organized collections of data. MySQL is a popular open-source database.'),
('MySQL Optimization Techniques', 'Learn indexing, query plans, and caching to speed up MySQL queries.'),
('Oracle vs MySQL', 'Comparison of Oracle and MySQL databases based on performance and cost.'),
('PostgreSQL Guide', 'PostgreSQL is an advanced open-source RDBMS.'),
('Data Science and SQL', 'SQL plays a key role in data science workflows.'),
('MySQL Tutorial for Beginners', 'This MySQL tutorial covers basic to intermediate topics for new developers.'),
('Cooking for Beginners', 'This guide is about basic cooking techniques and not related to databases.');

Boolean Mode - Practical Exercise

PES UNIVERSITY

1. Retrieve all articles that mention the word **“MySQL”** using natural language mode.
2. Get the relevance score for the term **"MySQL"**.
3. Find articles that **must include "MySQL"** but **must not include "Oracle"**.
4. Find articles that contain **either "tutorial" or "guide"**.
5. Find articles that **must contain “data”** and any word that starts with **"scien"** (like "science", "scientific").

(Refer notes for solutions!)

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu